

ASS 16.3

2303A52346

Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
- List all appointments for a specific doctor.
- Retrieve patient history by patient ID.
- Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

```
1 CREATE TABLE Doctors (  
2     doctor_id INT PRIMARY KEY AUTO_INCREMENT,  
3     doctor_name VARCHAR(100) NOT NULL,  
4     specialization VARCHAR(100),  
5     phone VARCHAR(15) UNIQUE,  
6     email VARCHAR(100) UNIQUE  
7 );  
8  
9 CREATE TABLE Patients (  
10     patient_id INT PRIMARY KEY AUTO_INCREMENT,  
11     patient_name VARCHAR(100) NOT NULL,  
12     dob DATE NOT NULL,  
13     gender ENUM('Male', 'Female', 'Other'),  
14     phone VARCHAR(15) UNIQUE,  
15     email VARCHAR(100) UNIQUE  
16 );
```

Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.

- Generate queries to:
- Retrieve all orders by a user.
- Find the most purchased product.
- Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'AI_AC_ASSIGNMENTS' with various files including '2346AI_AC_Assignment.py', 'aas 10 ai', 'AI ASS 5.5.pdf', 'AI ASSISTED CODING 6.3.pdf', 'AI ASSISTED CODING 1.5.pdf', 'AI-Assited_coding_assignment_3.5.pdf', 'ASS 2.5 2062 AI ASSISTED CODING (1).pdf', 'ASS 2.5 2346 AI ASSISTED CODING.pdf', 'ASS 4.5 AI ASSISTED 2346.pdf', 'aas 7', 'ASS 8', 'ASS 8.3 .pdf', 'ass 11.3', 'ASS 13.5.PY', 'ass9_2346.py', 'assign_grade.py', 'aas 9 ai', 'dynamo.db', 'dynamo.sql', 'exam_registration.py', 'lab exam', 'prjct', 'README.md', 'requirements.txt', 'run_tests.py', 'tes'. The code editor shows the following Python code:

```

1 class ArrayContactManager:
2     def __init__(self):
3         self.contacts = []
4
5     def add_contact(self, name, phone):
6         self.contacts.append({"name": name, "phone": phone})
7         print("Contact added successfully.")
8
9     def search_contact(self, name):
10        for contact in self.contacts:
11            if contact["name"] == name:
12                print("Contact found:")
13                print(f"Name: {contact['name']}, Phone: {contact['phone']}")
14                return
15        print("Contact not found.")
16
17    def delete_contact(self, name):
18        for i, contact in enumerate(self.contacts):
19            if contact["name"] == name:
20                del self.contacts[i]
21                print("Contact deleted successfully.")
22                return
23        print("Contact not found.")
24
25    def display_contacts(self):
26        if not self.contacts:
27            print("No contacts available.")
28            return
29
30        print("\n--- Contact List (Array) ---")
31        for i, contact in enumerate(self.contacts, start=1):
32            print(f"{i}. Name: {contact['name']}, Phone: {contact['phone']}")

```

Library Database

Task:

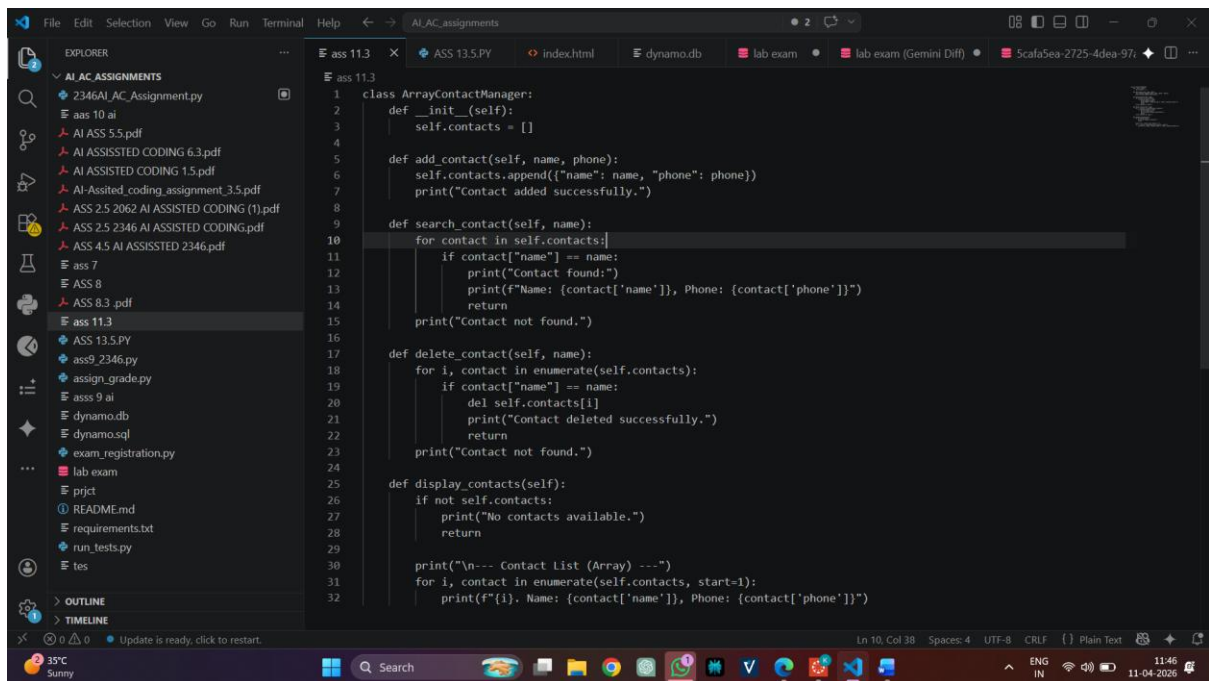
Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
- Retrieve all books currently issued.
- Find overdue books (loan date > 30 days).
- Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions



Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

-- Original query

```
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM
Authors WHERE country='UK');
```

-- Optimized query

```
SELECT b.*
```

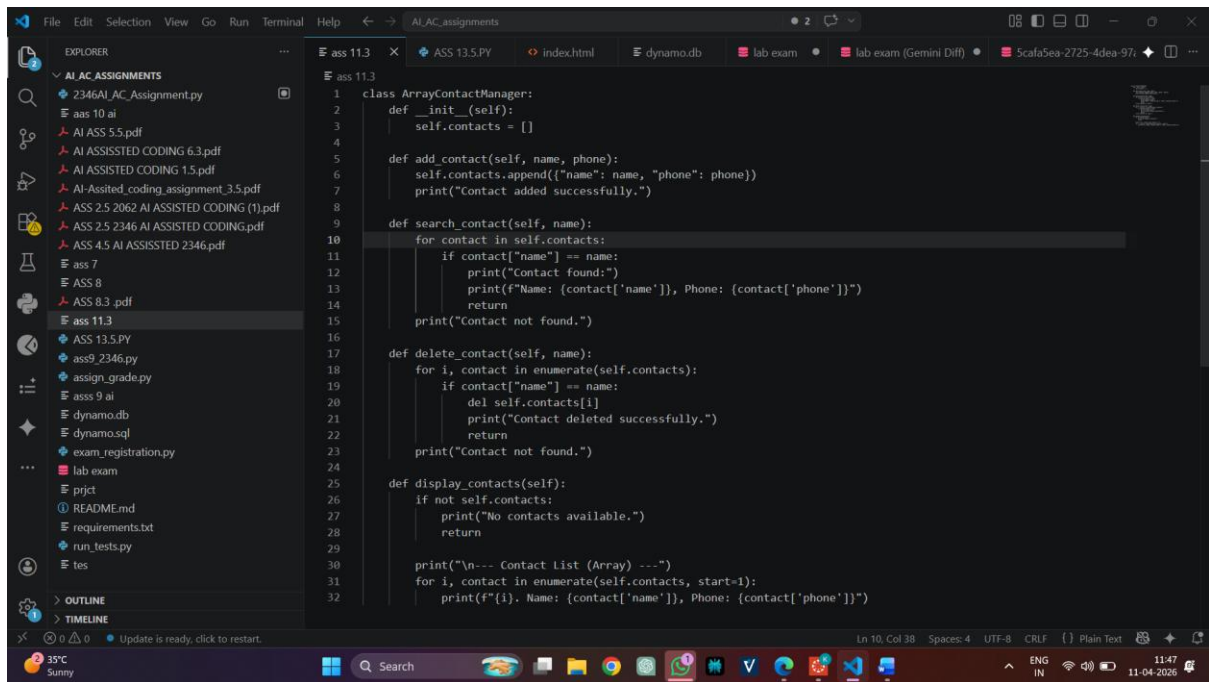
```
FROM Books b
```

```
JOIN Authors a ON b.author_id = a.author_id
```

```
WHERE a.country = 'UK';
```

Expected Output:

- Both queries return the same result: all books written by UK authors.
- Optimized query performs faster for larger datasets.



University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
- Each student can register for multiple courses.
- Each course is taught by a faculty member.
- Write SQL queries to:
- List all students enrolled in a specific course.
- Find faculty members teaching more than 2 courses.
- Retrieve courses with the highest number of registrations.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

